

TABLE OF CONTENTS

SL. NO	Table of Contents	Page No
1	List of Figures • List of Tables • List of Code Listings	ii
2	Introduction • Overview of Problem Domain • Objectives	1 - 2
3	Key Features and Analysis • Feature Analysis Table	3 - 5
4	Technology Stack • Development Environment	6 - 7
5	Requirements & Specifications • Functional • Non-Functional • Hardware/Software	8 - 10
6	User Interface Screenshots • Navigation Flow	11 - 18
7	Implementation • Architecture • Database • Modules • Core Logic	19 - 26
8	Conclusion • Contributions • Learning Outcomes • Challenges • Future Scope • References	27 - 29

LIST OF FIGURES

Figure	Title
Figure 1	Login screen
Figure 2	Registration screen
Figure 3	Student Dashboard
Figure 4	My Progress
Figure 5	Teacher Dashboard
Figure 6	Manage Words
Figure 7	Manage Sentences
Figure 8	Class Progress
Figure 9	Class Statistics
Figure 10	Pronunciation practice interface (mock-up)
Figure 11	Screen navigation flow
Figure 12	Layered system architecture
Figure 13	Local SQLite schema

LIST OF TABLES

Table	Title
Table 1	Feature analysis — features, descriptions, and technologies
Table 2	Technology stack
Table 3	Hardware and software requirements

LIST OF CODE LISTINGS

Listing	Title
Code 1	TTS Helper — playing the model pronunciation
Code 2	Speech Helper — launching voice recognition
Code 3	Pronunciation Activity — Levenshtein similarity scoring
Code 4	Sync Manager — connectivity check for offline queueing

1. INTRODUCTION

1.1 Overview of the Problem Domain

A large share of students in Karnataka's government schools study in Kannada-medium classrooms, yet English remains the language of higher education, employment, and digital services. For young learners in the 1st and 2nd standard, the transition to English is difficult because most learning material assumes prior English exposure and offers no support in the child's mother tongue. The gap is widest in spoken English and pronunciation, where students have very few opportunities to hear correct sounds and even fewer to practise speaking and receive feedback.

Basha Setu is an Android application built to bridge exactly this gap. It uses Kannada as a bridge language to introduce basic English vocabulary, pronunciation, and simple sentence formation, aligned with the difficulty level of the Karnataka Government School syllabus for the 1st and 2nd standards. Instead of treating English as an abstract subject, the app teaches through familiar, everyday categories such as animals, fruits, daily-use objects, school objects, and numbers, so that new words are anchored to things the child already recognises.

The core learning loop is built around three actions - Listen, Speak, and get Feedback. The learner hears the correct pronunciation of a word or sentence through the device's Text-to-Speech engine, repeats it aloud, and the app captures the spoken input through speech recognition and scores it, returning an instant, encouraging rating. This makes independent, repetition-based pronunciation practice possible even without a fluent English-speaking teacher in the room.

The problem domain of the project therefore encompasses the following key challenges:

- Delivering age-appropriate, syllabus-aligned English content to very young learners in a simple, visual, and encouraging way.
- Using Kannada (and optionally Hindi) alongside English so that comprehension is never blocked by the language barrier.
- Providing objective, real-time pronunciation feedback using on-device speech recognition rather than relying only on a teacher.
- Working reliably in low-connectivity government-school environments, which requires an offline-first design with local storage and background synchronisation.
- Supporting two distinct roles — students, who learn and practise, and teachers, who manage content and monitor class performance.

1.2 Objectives

The primary objectives that guided the design and development of Basha Setu are:

- To build a mobile-first, offline-capable English learning tool tailored to Kannada-medium government-school children.
- To make correct English pronunciation audible and practisable through Text-to-Speech and speech recognition.
- To provide objective, encouraging feedback and light gamification (XP, streaks, daily goals) that motivates young learners.

2. KEY FEATURES AND ANALYSIS

The Basha Setu application is organised into a set of major functional modules that together deliver a complete, role-aware, offline-capable English learning experience. Each module was developed and integrated into the single Android application (package com.example.pbl).

2.1 Authentication and Role Selection

- Email/password sign-in and one-tap Google Sign-In, both backed by Firebase Authentication.
- Role-based access at login: a user registers and signs in either as a Student or as a Teacher, which decides the home screen and available features.
- Students additionally select their class Standard (1st or 2nd), which filters all learning content to the correct syllabus level.

2.2 Standard-Based Learning

- Learners choose between 1st Standard and 2nd Standard; all vocabulary, sentences, and pronunciation exercises are scoped to that level.
- Content difficulty is structured to match syllabus progression, ensuring age-appropriate learning.

2.3 Category-Based Vocabulary

- Learning is grouped into familiar categories: Animals, Fruits, Daily-Use Objects, School Objects, Numbers, and Simple Sentences.
- Each item stores its English text along with its Kannada and Hindi equivalents and an associated image, so meaning is conveyed visually and in the learner's own language.

2.4 Interactive Pronunciation Practice

- Listen: the learner hears the correct pronunciation through the Android Text-to-Speech (TTS) engine, with adjustable speech speed.
- Speak: the learner repeats the word or sentence, captured through the Android SpeechRecognizer (RecognizerIntent) voice input.
- Feedback: the spoken input is compared with the target and scored, giving instant tiered feedback — Excellent, Great Job, Good Try, or Keep Practising — with a percentage score.

2.5 Progress Tracking and Gamification

- Every attempt is recorded with its score, category, and timestamp in a local SQLite database and synced to the cloud.
- Learners earn XP points, maintain daily streaks, and work toward a simple daily goal, all shown on the home dashboard.
- A Progress screen summarises performance with overview statistics, per-category progress, and insights such as the hardest category.

2.6 Teacher Tools

- A dedicated Teacher Dashboard gives teachers a separate entry point after login.
- Manage Content lets teachers add, edit, and delete words and sentences (with images) that are pushed to all students through the cloud.
- Class Statistics presents a class leaderboard and highlights the category students find hardest, helping teachers target instruction.

2.7 Offline-First Synchronisation

- All learning content, user profiles, settings, and progress are cached locally in SQLite so the app works fully offline.
- A SyncManager detects connectivity, queues changes made while offline, and merges local and cloud data using last-sync-time logic when the device reconnects.

2.8 Feature Analysis Table

Table 1 summarises the application's major features, a brief description of each, and the primary technology used to implement it. Together these features cover the full learner and teacher workflow described in the sections above.

Table 1: Feature analysis — features, descriptions, and technologies used.

Feature	Description	Technology Used
Authentication	Student/Teacher login with email or Google	Firebase Auth, Google Sign-In
Standard Selection	Filter content by 1st / 2nd standard	Java, Firestore, SQLite
Vocabulary Learning	Category-wise words with Kannada/Hindi + image	RecyclerView, Glide, Cloudinary
Pronunciation Practice	Listen, speak and score spoken input	Android TTS, SpeechRecognizer
Scoring Engine	Fuzzy match of speech vs target word	Levenshtein distance algorithm
Progress & Gamification	XP, streaks, daily goal, per-category stats	SQLite, Firestore
Teacher Content Mgmt	Add / edit / delete learning content	Firestore, Cloudinary, Glide
Class Statistics	Leaderboard and hardest-category insight	Firestore, Java
Offline Sync	Local cache with background merge	SQLite, SyncManager, Firestore

3. TECHNOLOGY STACK

Table 2 lists the technologies used to build the Basha Setu Android application, organised by layer, along with the version or configuration detail for each. The stack combines a Java/Android front end with a Firebase cloud back end and an on-device SQLite cache.

Table 2: Technology stack used in the development of Basha Setu.

Layer	Technology	Version / Details
Programming Language	Java	Java 11 (Android compatible)
Platform	Android SDK	Min SDK 28 (Android 9.0), Target/Compile SDK 36
IDE	Android Studio	with embedded Gemini / Studio Bot
Build System	Gradle	Gradle with Kotlin DSL (.kts)
UI Framework	Material Design 3	com.google.android.material
Layouts	ConstraintLayout, RecyclerView	AndroidX
Authentication	Firebase Auth + Google Sign-In	play-services-auth, Firebase BoM
Cloud Database	Firebase Firestore	Content, users, and progress sync
Cloud Storage	Firebase Storage + Cloudinary	cloudinary-android 3.1.2 for images
Local Database	SQLite (SQLiteOpenHelper)	'SmartPronunciation.db', schema v4
Image Loading	Glide	with Glide annotation processor
Speech Output	Android Text-to-Speech (TTS)	android.speech.tts
Speech Input	Android SpeechRecognizer	RecognizerIntent voice recognition

3.1 Development Environment

- IDE: Android Studio
- Emulator / Device: Android Studio Emulator and physical Android device (Android 9.0+)
- Build Tool: Gradle (Kotlin DSL)
- Cloud Back-end: Firebase project (Authentication, Firestore, Storage) configured via google-services
- Permissions: RECORD_AUDIO (speech input), INTERNET (sync), READ_MEDIA_IMAGES (content images)

4. REQUIREMENTS AND SPECIFICATIONS

• 4.1 Functional Requirements with Input, Process and Output

FR-1: User Registration and Login

- Input:
- Email ID and Password or Google Sign-In
- User Role (Student/Teacher)
- Process:
- Validate user credentials.
- Authenticate the user.
- Store user details securely.
- Output:
- User successfully logs in and is redirected to the appropriate dashboard.

FR-2: Class Selection

- Input:
- Student selects Standard (1st or 2nd).
- Process:
- Store the selected class.
- Load learning content related to the selected class.
- Output:
- Learning modules for the selected class are displayed.

FR-3: Dashboard Navigation

- Input:
- Authenticated user role.
- Process:
- Check whether the user is a Student or Teacher.
- Redirect to the respective dashboard.
- Output:
- Student Home Screen or Teacher Dashboard is displayed.

FR-4: Learning Content

- Input:
- Selected learning category (Animals, Fruits, Numbers, School, etc.)
- Process:

- Retrieve vocabulary from the database.
- Display text, image, and translation.
- Output:
- Vocabulary item with image and Kannada/Hindi meaning is shown.

FR-5: Pronunciation Audio

- Input:
- User selects a vocabulary word.
- Process:
- Convert text into speech using Text-to-Speech (TTS).
- Output:
- Correct pronunciation of the selected word is played.

FR-6: Speech Recognition

- Input:
- Student speaks the displayed word.
- Process:
- Capture voice using Speech Recognition.
- Convert speech into text.
- Output:
- Recognized spoken text is displayed for evaluation.

FR-7: Pronunciation Evaluation

- Input:
- Target word and recognized spoken word.
- Process:
- Compare both words.
- Calculate pronunciation similarity percentage.
- Output:
- Pronunciation score (%) is generated.

FR-8: Feedback Generation

- Input:
- Pronunciation score.
- Process:

- Compare score with predefined thresholds.
- Output:
- Feedback such as Excellent, Great Job, Good Try, or Keep Practising is displayed.

FR-9: Progress Tracking

- Input:
- Student's pronunciation score.
- Process:
- Update XP points, streaks, and progress records.
- Output:
- Updated progress and achievements are displayed.

FR-10: Teacher Content Management

- Input:
- New vocabulary, image, category, or translation.
- Process:
- Validate data.
- Add, edit, or delete content from the database.
- Output:
- Learning content is updated successfully.

FR-11: Leaderboard

- Input:
- Students' learning scores.
- Process:
- Rank students based on XP and performance.
- Output:
- Leaderboard with rankings is displayed.

FR-12: Offline Synchronization

- Input:
- Learning progress and internet availability.
- Process:
- Save progress locally.
- Synchronize data with the cloud when internet is available.
- Output:

- User progress is safely stored and updated.

4.2 Non-Functional Requirements

NFR-1: Offline Capability

- Input:
- User opens the application without an internet connection.
- Process:
- The application loads locally stored learning content and user progress from the device database.
- Any new progress is saved locally until an internet connection becomes available.
- Output:
- The user can continue learning normally in offline mode, and progress is stored safely.

NFR-2: Reliability

- Input:
- User progress, settings, and application data.
- Process:
- Save all data automatically in local storage.
- Restore the saved data when the application restarts.
- Synchronize local data with the cloud when internet connectivity is restored.
- Output:
- User data is preserved without loss, even after app restarts or temporary network failures.

NFR-3: Usability

- Input:
- User interactions such as button clicks, screen navigation, and learning activities.
- Process:
- Display a child-friendly interface with large buttons, colorful visuals, readable fonts, and simple navigation suitable for 1st and 2nd standard students.
- Output:
- Users can easily understand and navigate the application with minimal guidance.

NFR-4: Performance

- Input:
- Student completes speaking a vocabulary word.
- Process:
- The application processes the recorded speech, compares it with the target word, and calculates the pronunciation score efficiently.
- Output:

- Pronunciation score and feedback are displayed quickly, providing a smooth learning experience.

NFR-5: Compatibility

- Input:
- Application installed on an Android device.
- Process:
- Verify Android version compatibility and ensure all application features work correctly on Android 9.0 (API 28) and above.
- Output:
- The application runs successfully across supported Android devices without compatibility issues.

NFR-6: Maintainability

- Input:
- New feature request, bug fix, or application update.
- Process:
- Organize the application using separate modules such as Activities (UI), Model classes, Helper classes (Text-to-Speech, Speech Recognition), and Data Managers (DBHelper, SyncManager), making the code easier to modify and maintain.
- Output:
- The application is easier to update, debug, and extend without affecting existing functionality.

4.3 Hardware and Software Requirements

Table 3 lists the hardware and software required both to develop the application and to run it on an end-user device.

Table 3: Hardware and software requirements for development and deployment.

Requirement	Development	Deployment (Device)
OS	Windows 10 / 11 or macOS	Android 9.0 (API 28) or higher
RAM	Minimum 8 GB	Minimum 2 GB
Storage	10 GB free disk space	~50 MB for app installation
Processor	Intel Core i5 or equivalent	Any ARM-based Android processor
Microphone	Not mandatory	Required for pronunciation practice
Internet	Required for Gradle & Firebase	Optional (needed only to sync)
IDE	Android Studio	Not applicable (end user)

5. USER INTERFACE SCREENSHOTS

5.1 Screenshots of Main App

Screens Screen 1 — Login

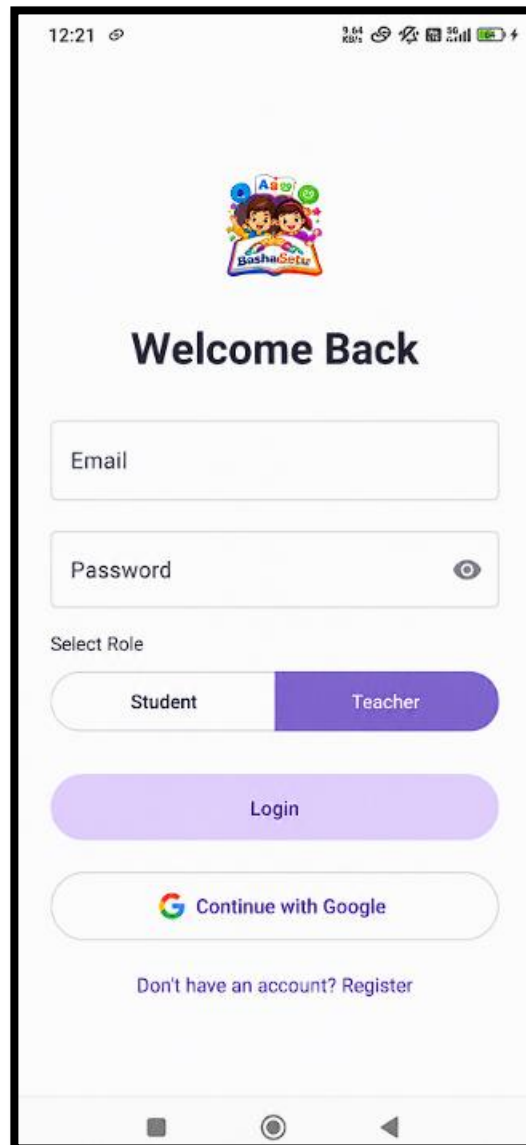


Figure 1: Students and teachers log in using an email/password combination or one-tap Google Sign-In. The role is chosen at sign-in.

Students and teachers log in using an email/password combination or one-tap Google Sign-In. A Student/Teacher role toggle sets which experience loads after sign-in. Includes a "Don't have an account? Register" link.

Screen 2 — Registration



Create Account

Full Name

Email

Password 

I am a:

☒ Student ☐ Teacher

Standard (e.g. 1, 2)

Register

 Continue with Google

[Already have an account? Login](#)

Figure 2: New users create an account and choose their role (Student or Teacher) before entering the app.

New users create an account by entering their full name, email, and password, and choosing a role (Student or Teacher). For students, this leads into the learning flow; for teachers, into the teacher tools.

Screen 3 — Student Dashboard

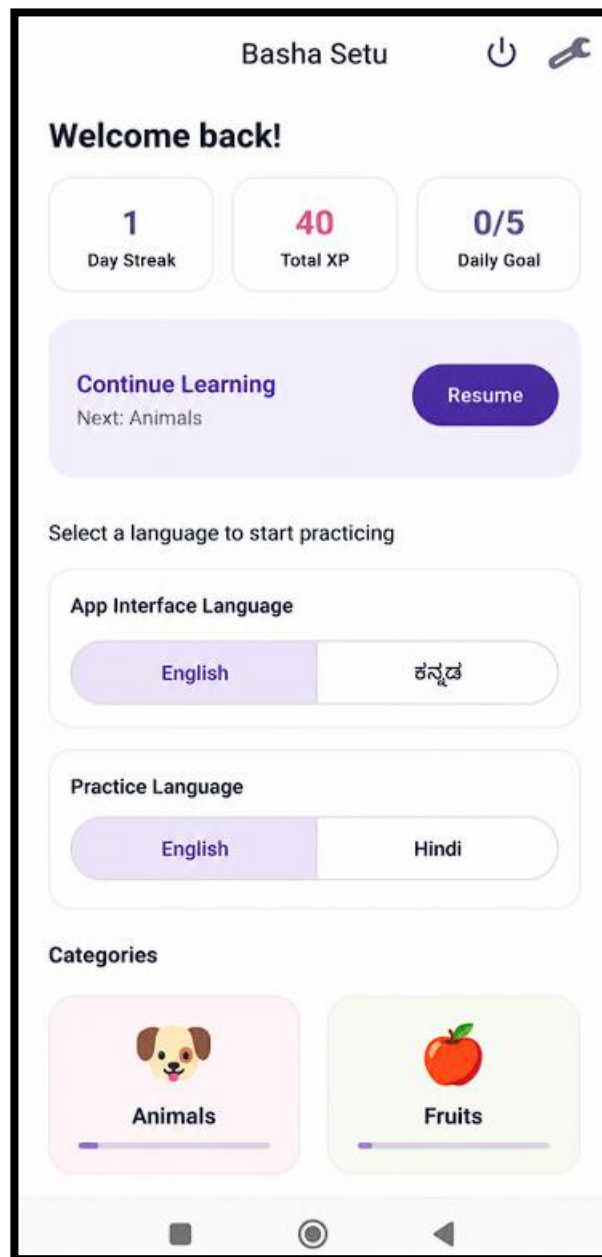


Figure 3: The learner's home screen: overall progress, XP, day streak, language selection, and the practice categories.

The learner's home screen. Shows a "Welcome back!" header with stat cards for Day Streak, Total XP, and Daily Goal, a "Continue Learning" resume shortcut, app-interface and practice-language toggles (English/Kannada, English/Hindi), and the practice categories (Animals, Fruits, and more) with progress bars.

Screen 4 — My Progress

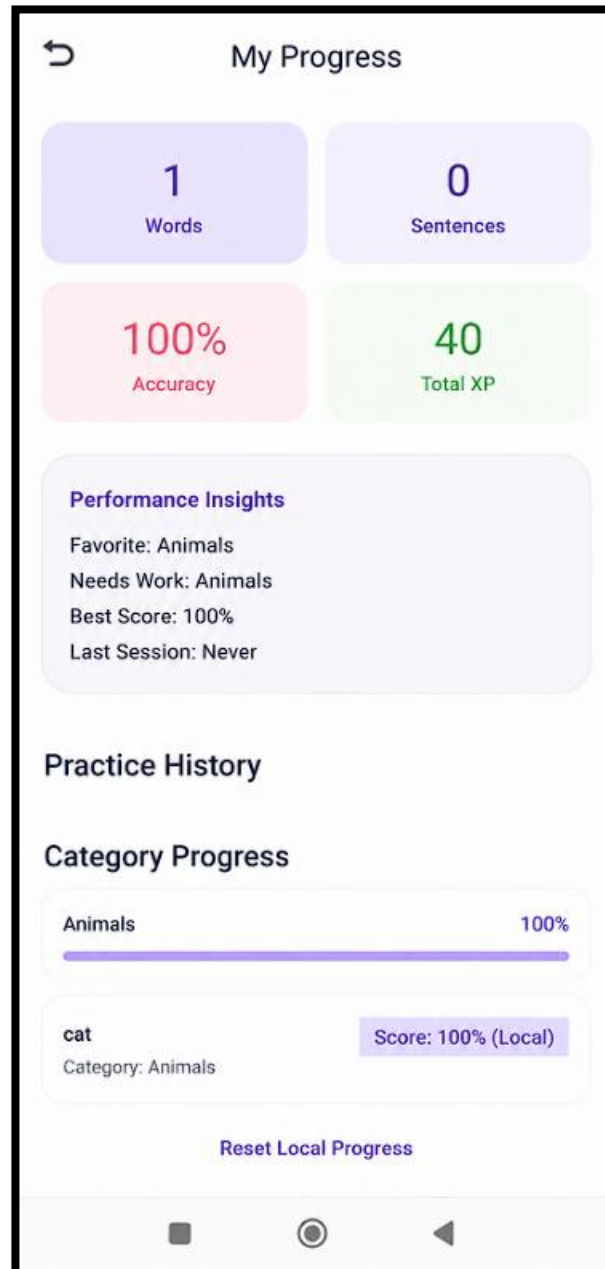


Figure 4: Learning statistics for the student: accuracy, XP earned, per-category progress, and recent practice history.

The student's learning statistics: overall accuracy, XP earned, per-category progress, and recent practice history, giving learners a clear view of how they are improving.

Screen 5 — Teacher Dashboard

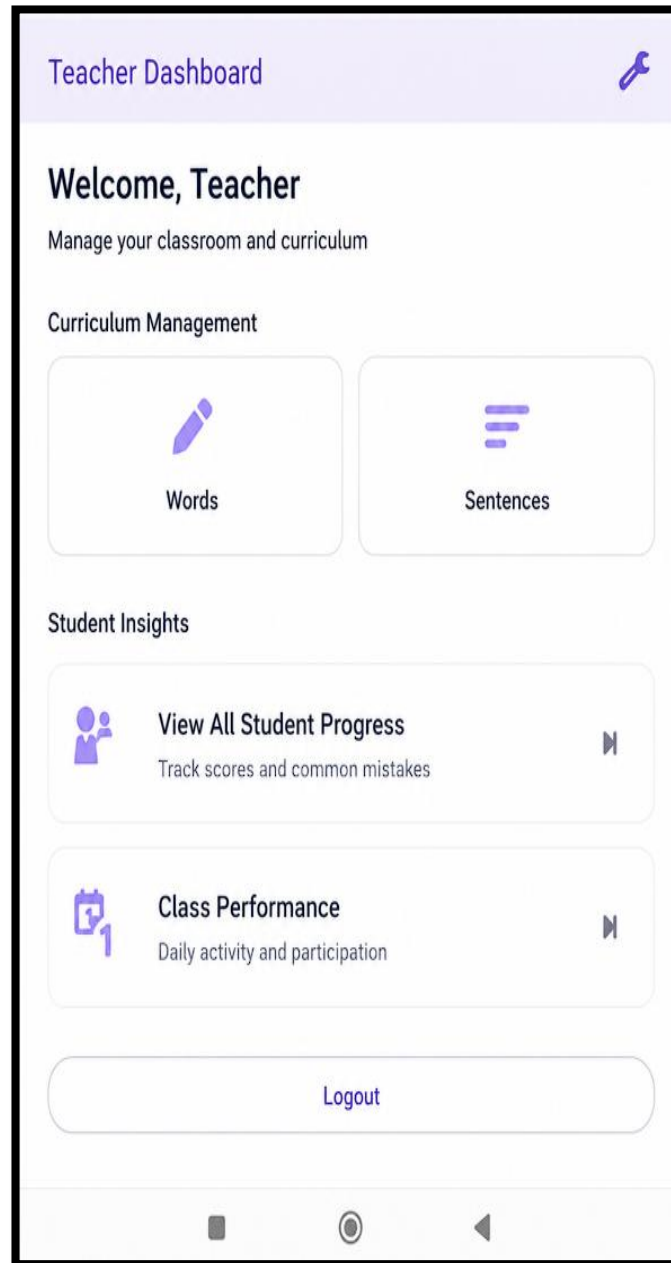


Figure 5: The teacher's home screen, providing curriculum-management options and student-performance tools.

The teacher's landing screen, providing curriculum-management options (Words and Sentences) and access to student-performance and class-analytics tools.

Screen 6 — Manage Words

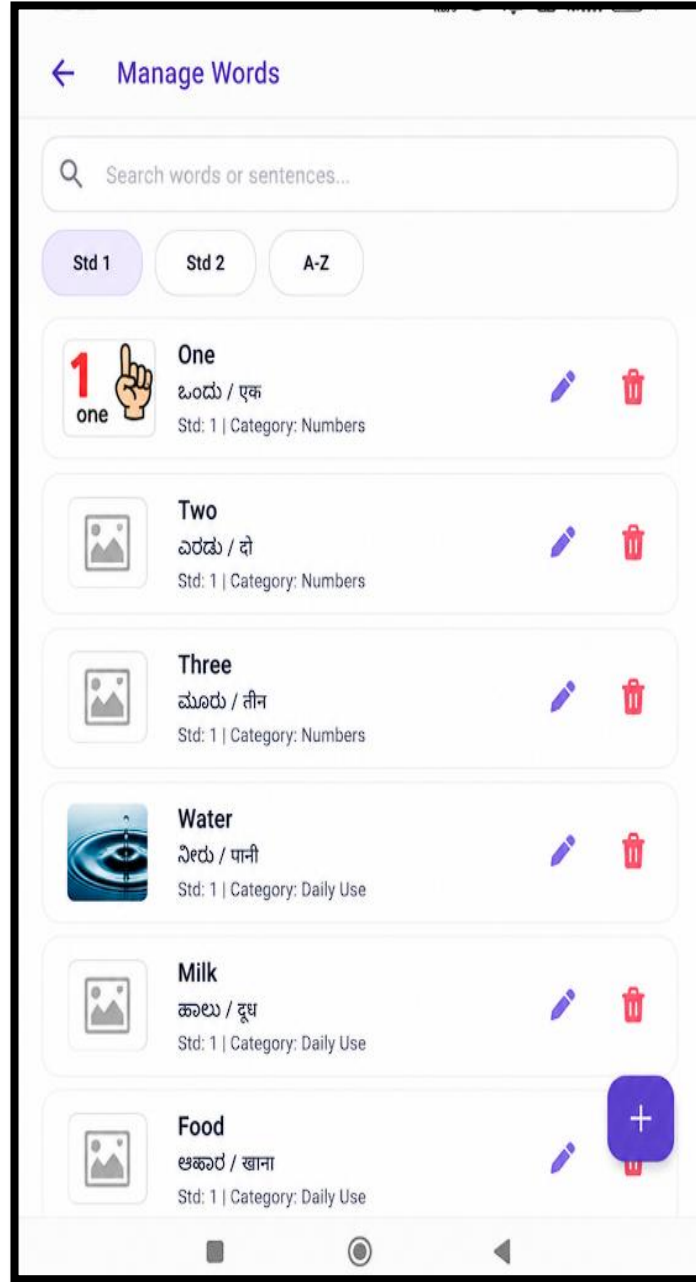


Figure 6: Teachers search, add, edit, and delete vocabulary words in the shared word bank.

Teachers search, add, edit, and delete vocabulary words in the shared word bank, with each entry linking an English word to its Kannada meaning.

Screen 7 — Manage Sentences

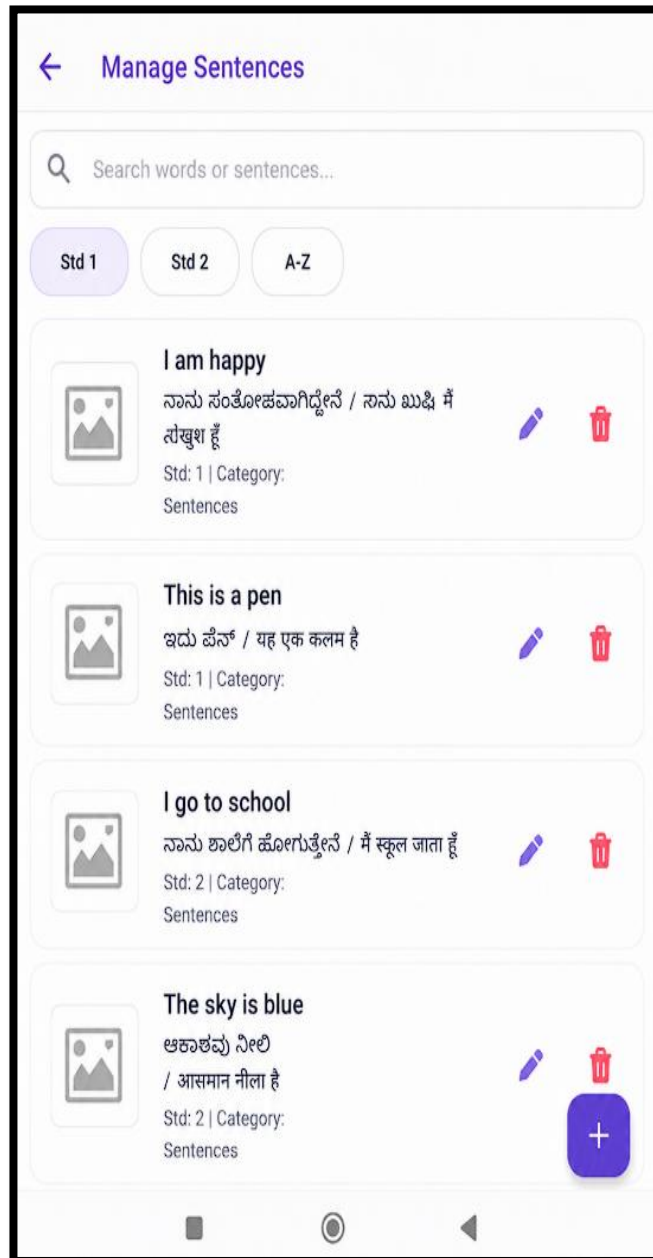


Figure 7: Teachers manage the bilingual practice sentences used in sentence exercises.

Teachers manage the bilingual practice sentences used in the sentence exercises — adding, editing, or removing them as needed.

Screen 8 — Class Progress

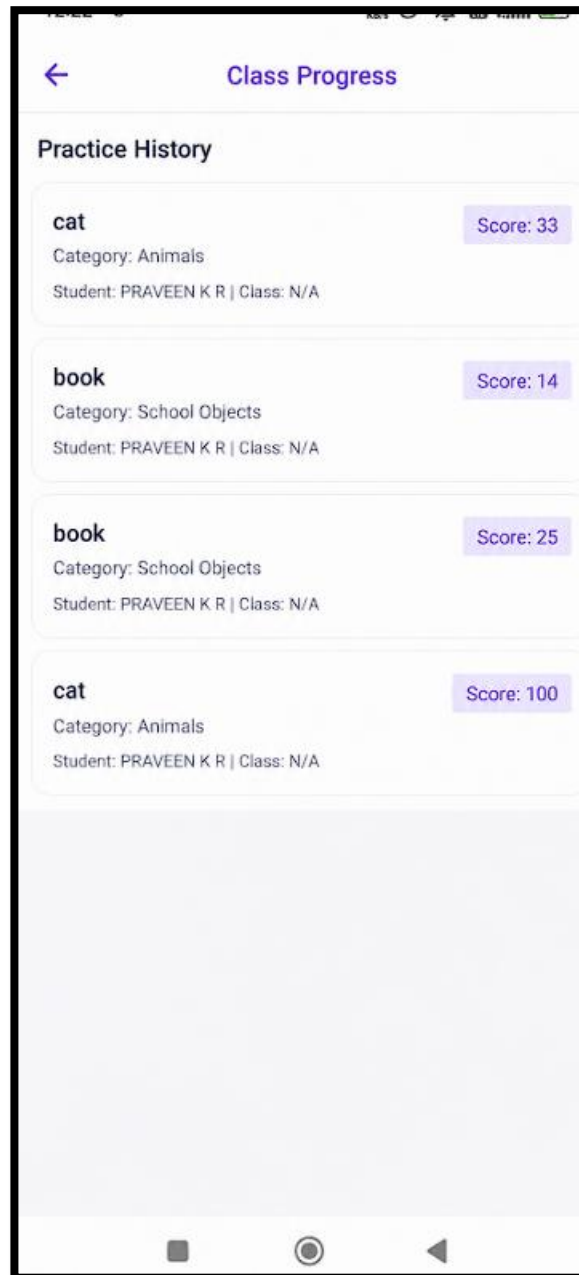


Figure 8: A teacher view of students' practice history, scores, and individual learning performance.

A teacher view of students' practice history, scores, and individual learning performance across the class.

Screen 9 — Class Statistics

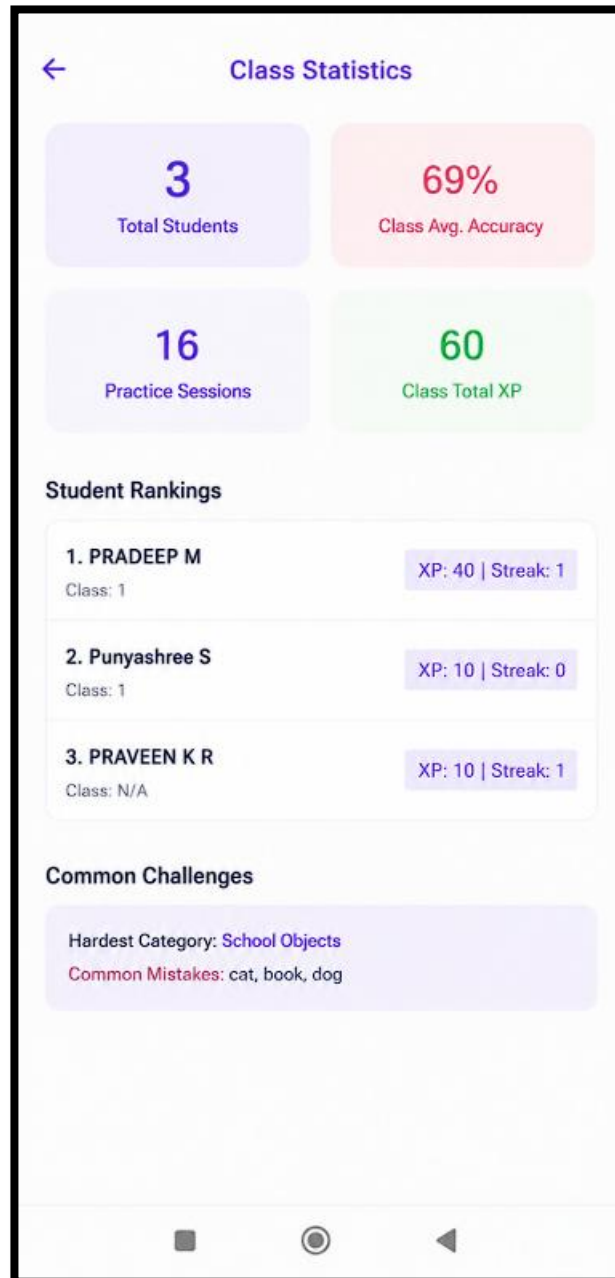


Figure 9: Class-level analytics: rankings, XP leaderboard, average accuracy, and common learning challenges.

Class-level analytics: an XP leaderboard with rankings, average accuracy, practice sessions, and the common learning challenges (such as the hardest category) for the class.

5.2 Pronunciation Practice (Interface Mock-up)

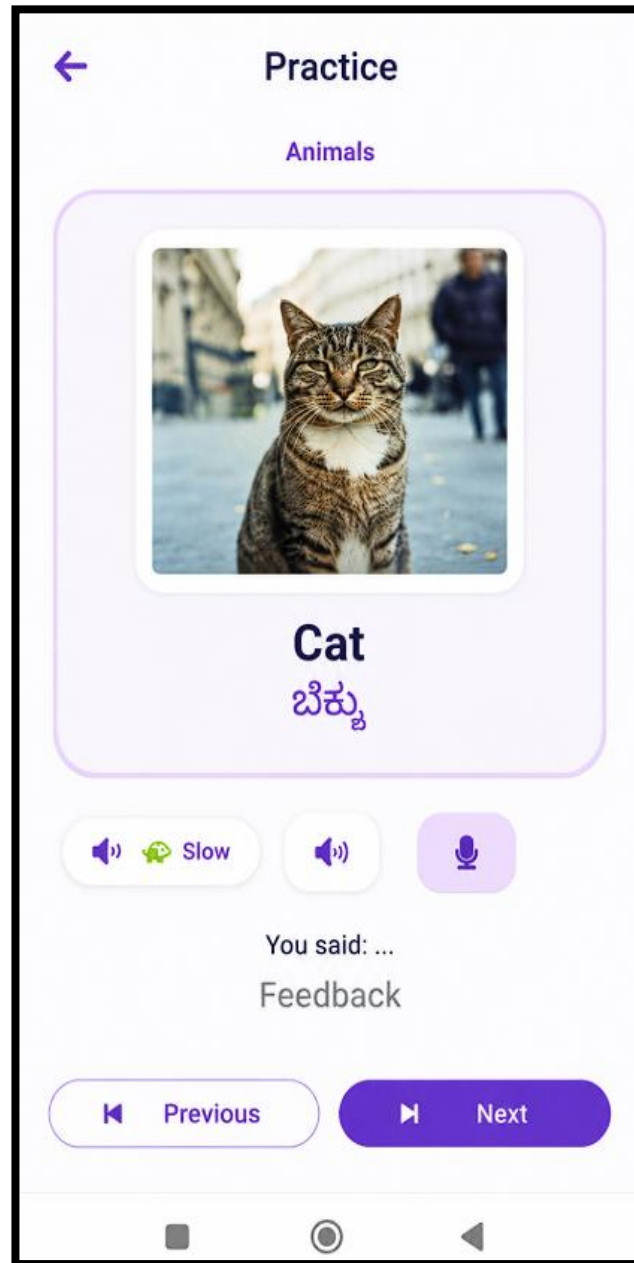


Figure 10: Pronunciation practice interface — the listen, speak, and feedback loop.

The pronunciation practice screen is the core learning loop of the application. As this screen was not part of the supplied screenshot set, the illustrative mock-up below represents its layout: the target word with its image and meaning, a Slow-speed option, Listen (Text-to-Speech) and Speak (microphone) controls, and instant scored feedback

5.3 Navigation Flow

The screens are connected through explicit Intent-based navigation. The high-level flow is illustrated below, followed by a step-by-step description.

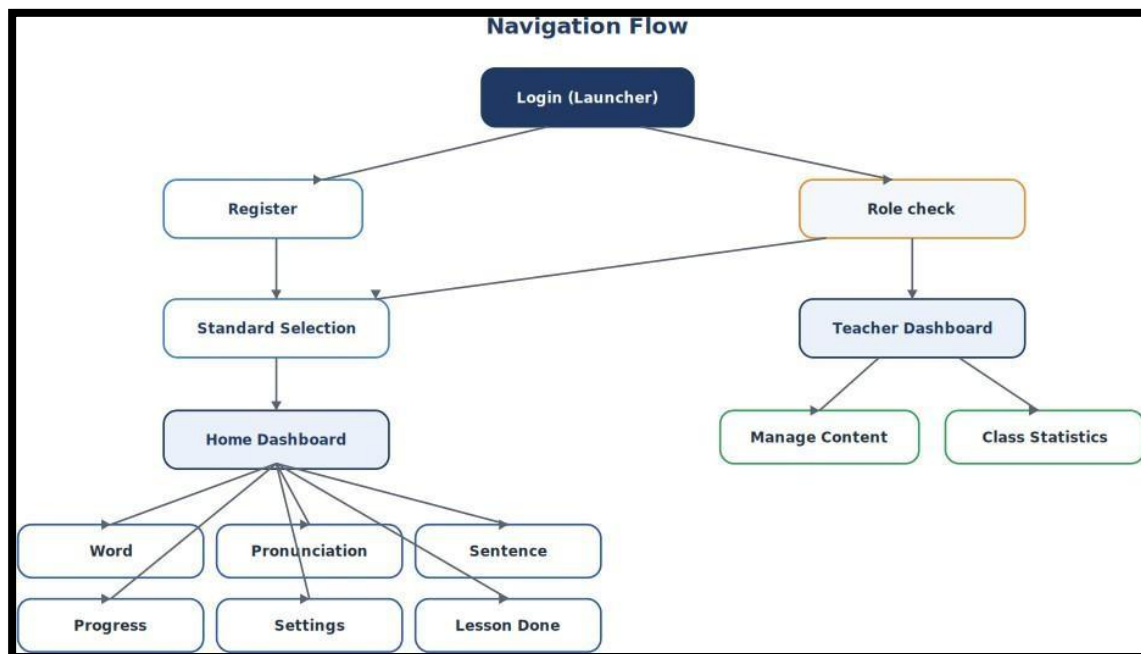


Figure 11: Screen navigation flow for the student and teacher paths.

- **1. Application Launch** — the app opens on the Login Activity (the launcher activity).
- **2. Authentication** — the user signs in as Student or Teacher via email/password or Google Sign-In. New users go to Register Activity first.
- **3. Student Path** — a student selects a class standard (Standard Selection Activity) and lands on the Main Activity home dashboard.
- **4. Learning** — from the dashboard the student opens a category and moves into Word Activity, Pronunciation Activity, or Sentence Activity to practise.
- **5. Progress & Settings** — Progress Activity and Settings Activity are reachable from the dashboard to review performance and adjust preferences.
- **6. Teacher Path** — a teacher signs in and lands on Teacher Dashboard Activity, from where Manage Content Activity and Class StatsActivity are opened.
- **7. Lesson Completion** — on finishing a set, the learner sees Lesson Completion Activity summarising the session before returning home.

6. IMPLEMENTATION DETAILS

The application follows a modular, offline-first design. UI logic lives in Activities, data models in dedicated classes (User, Word), reusable behaviour in helper classes (TTSHelper, Speech Helper), and all persistence and synchronisation in DB Helper and Sync Manager.

6.1 System Architecture

Basha Setu is organised into four cooperating layers on the device, as shown in Figure 12 — a Presentation layer of Activities, a Domain/Helper layer of reusable services, a local Data layer built on SQLite, and a Cloud back-end on Firebase and Cloudinary. The device works primarily against the local layer for speed and offline support, while the Sync Manager reconciles that local data with the cloud whenever connectivity is available.

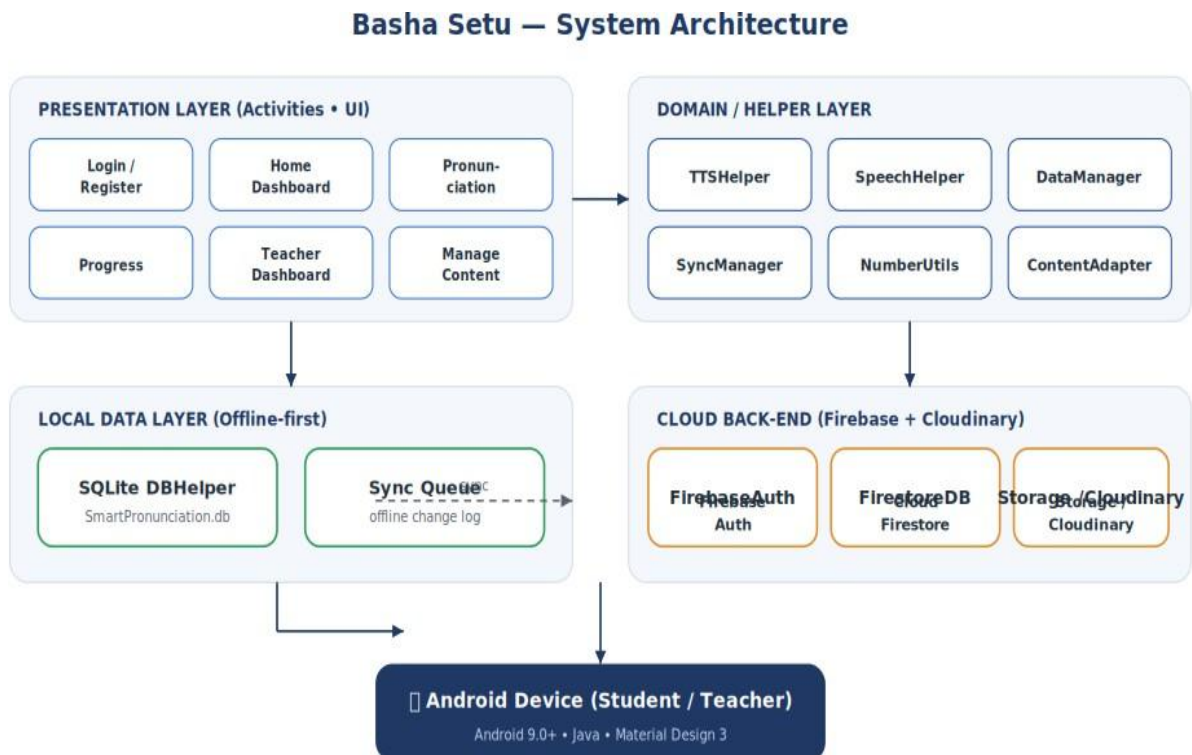


Figure 12: Layered system architecture of Basha Setu.

6.2 Database Design

The on-device SQLite database, 'SmartPronunciation.db' (schema version 4), is the single source of truth while offline. As shown in Figure 13, it holds five tables: users (profile, XP, streak, standard and cached settings), content (English/Kannada/Hindi text, image URL and type), progress (every scored attempt), preferences (personalisation), and sync_queue (a log of offline attempts awaiting upload).

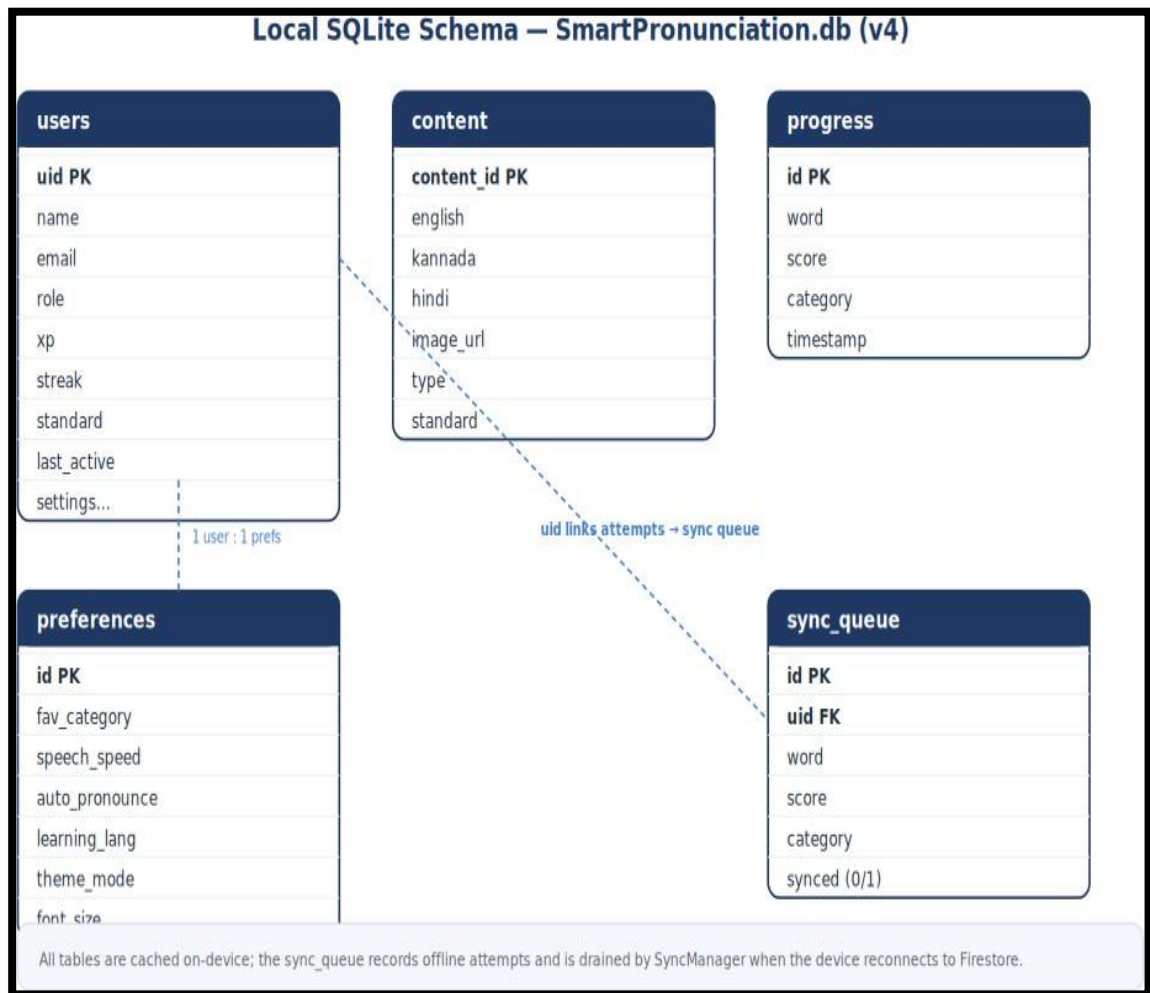


Figure 13: Local SQLite schema (SmartPronunciation.db).

6.3 – 6.11 Module Descriptions

The core modules and their responsibilities are described below.

6.3 Module 1 — Authentication & Roles (LoginActivity, RegisterActivity)

Handles user identity and role-based routing across the whole app.

Functions:

- Authenticate users with Firebase Auth using email/password or Google Sign-In.
- Let the user choose a Student or Teacher role and, for students, a class standard.
- Store the user profile in Firestore and cache it locally for offline access.
- Route the user to the correct home screen based on their role.

6.4 Module 2 — Student Home Dashboard (MainActivity)

The central hub for a learner, summarising progress and launching lessons.

Functions:

- Load the signed-in user and show streak count, XP, and daily-goal progress.
- Compute and refresh per-category progress from the local database.
- Offer a 'resume last category' shortcut to continue where the learner stopped.
- Provide navigation to Progress, Settings, and the learning categories.

6.5 Module 3 — Vocabulary & Sentences (WordActivity, SentenceActivity, ContentAdapter)

Presents category-wise words and sentences with multilingual support.

Functions:

- Load content for the selected standard and category from cache or Firestore.
- Display each item's English text with its Kannada/Hindi meaning and image (Glide).
- Use a RecyclerView with ContentAdapter to render the scrollable item list.
- Feed selected items into the pronunciation practice flow.

6.6 Module 4 — Pronunciation & Scoring Engine (PronunciationActivity, TTSHelper, SpeechHelper)

The core learning loop: hear, speak, and get scored feedback.

Functions:

- Speak the target word aloud using the Text-to-Speech engine at the chosen speed.
- Capture the learner's speech through the SpeechRecognizer and normalise it (including spoken numbers via NumberUtils).
- Score the attempt using a Levenshtein-distance fuzzy match against the target.
- Show tiered feedback, award XP for passing scores, and record the attempt locally.

6.7 Module 5 — Progress & Gamification (ProgressActivity, LessonCompletionActivity)

Tracks and visualises the learner's improvement over time.

Functions:

- Aggregate stored attempts into overview statistics and per-category progress.

- Maintain XP totals, daily streaks, and daily-goal completion.
- Show a lesson-completion summary at the end of each practice set.
- Surface simple insights such as the learner's strongest and weakest categories.

6.8 Module 6 — Settings (SettingsActivity)

Lets learners personalise how the app looks and behaves.

Functions:

- Adjust speech speed, auto-pronounce, and the learning language.
- Switch theme mode and font size for readability.
- Toggle leaderboard participation and daily goals.
- Persist all preferences locally and sync them to the cloud.

6.9 Module 7 — Teacher Content Management (TeacherDashboardActivity, ManageContentActivity)

Gives teachers control over the learning material students receive.

Functions:

- Provide a teacher-only dashboard after role-based login.
- Add, edit, and delete words and sentences with associated images.
- Upload images via Cloudinary and store content records in Firestore.
- Push updated content to all students through cloud synchronisation.

6.10 Module 8 — Class Statistics (ClassStatsActivity)

Helps teachers monitor and act on class-wide performance.

Functions:

- Aggregate student XP into a class leaderboard.
- Identify the category students find hardest across the class.
- Read from Firestore with a local fallback for reliability.

6.11 Module 9 — Data & Sync Layer (DBHelper, DataManager, SyncManager)

The offline-first backbone shared by every other module.

Functions:

- Define and manage the local SQLite schema (users, content, progress, preferences, sync queue).
- Seed default built-in content through DataManager when the cloud is unreachable.
- Detect connectivity, queue offline changes, and merge local/cloud data on reconnection.
- Guarantee that progress and settings are never lost between sessions.

6.12 Core Logic and Code Snippets

This section illustrates the core logic with four short code listings (Code 1 to Code 4) drawn from the project: playing the model pronunciation, capturing the learner's speech, scoring the attempt, and deciding between a cloud write and an offline queue.

(a) Speaking the word — Text-to-Speech

The TTSHelper wraps Android's TextToSpeech engine. It sets the language, applies the learner's chosen speech rate, and speaks the target word so the child hears a correct model before attempting it.

```
public void speak(String text) {
    if (isInitialized) {
        tts.speak(text, TextToSpeech.QUEUE_FLUSH, null, null);
    }
}

public void setSpeechRate(float rate) {
    if (tts != null) { tts.setPitch(1.0f); tts.setSpeechRate(rate); }
}
```

Code 1: TTSHelper — playing the model pronunciation at an adjustable speed.

(b) Capturing speech — SpeechRecognizer

When the learner taps Speak, SpeechHelper launches the system speech recogniser with the correct language (English or Hindi) and tuned silence timeouts, and the recognised text is returned to the Activity for scoring.

```
Intent intent = new Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH);
intent.putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL,
    RecognizerIntent.LANGUAGE_MODEL_FREE_FORM);
String lang = isHindi ? "hi-IN" : "en-US";
intent.putExtra(RecognizerIntent.EXTRA_LANGUAGE, lang);
activity.startActivityForResult(intent, SPEECH_REQUEST_CODE);
```

Code 2: SpeechHelper — launching voice recognition for the learner's attempt.

(c) Scoring the attempt - fuzzy match

The spoken text is lower-cased and normalised, then compared to the target using the Levenshtein edit distance. The ratio of the distance to the longer string is inverted to give a 0–100 similarity score, which drives both the feedback tier and the XP awarded.

```
private int calculateFuzzyScore(String target, String input) {
    if (target.isEmpty() || input.isEmpty()) return 0;
    if (target.equals(input)) return 100;
    int distance = LevenshteinDistance(target, input);
    int maxLength = Math.max(target.length(), input.length());
    float score = (1.0f - (float) distance / maxLength) * 100;
    return Math.max(0, Math.round(score));
}
```

Code 3: Pronunciation Activity — Levenshtein-based similarity scoring.

Feedback tiers are then assigned from the score: 95% and above shows "Excellent", 80% and above "Great Job", 60% and above "Good Try", and below 60% "Keep Practising". A passing score of 60% or more awards XP equal to one-tenth of the score, and a score of 80% or more marks the word as mastered.

(d) Offline-first persistence and sync

Every attempt is written to the local progress table immediately. If the device is online the score is also pushed to Firestore; if it is offline the attempt is added to the sync_queue and drained later. Connectivity is checked with a lightweight helper:

```
public boolean isOnline() {  
    ConnectivityManager cm = (ConnectivityManager)  
        context.getSystemService(Context.CONNECTIVITY_SERVICE);  
    NetworkInfo netInfo = cm.getActiveNetworkInfo();  
    return netInfo != null && netInfo.isConnected();  
}
```

Code 4: SyncManager — connectivity check that decides between cloud write and offline queue.

7. CONCLUSION

The Basha Setu Android application was successfully designed and implemented as a collaborative MCA project. It delivers a complete, offline-capable English learning tool for government-school students, using Kannada as a bridge language to teach vocabulary, pronunciation, and simple sentences. Learners practise through a listen–speak–feedback loop backed by on-device text-to-speech and speech recognition, while teachers manage content and monitor class performance. The project demonstrates the practical application of Android SDK concepts, Firebase back-end integration, local SQLite persistence, offline synchronisation, external APIs, and Material Design UI.

7.1 Summary of Contributions

Each team member led one or more modules of the application:

Punyashree S (1RV25MC077) - led the Pronunciation & Scoring Engine, including the Text-to-Speech and speech-recognition integration and the Levenshtein-based scoring logic.

Pradeep M (1RV25MC069) - built Authentication & Roles and the Student Home Dashboard, wiring up Firebase Auth, Google Sign-In, and role-based navigation.

Praveen K R (1RV25MC073) - developed the Vocabulary & Sentences modules and Progress & Gamification, including the RecyclerView content lists and XP/streak tracking.

Darshan S R (1RV25MC024) - implemented Teacher Content Management, Class Statistics, and Settings, plus the offline Data & Sync layer (DBHelper and SyncManager).

7.2 Learning Outcomes

Through the development of this project, the team gained hands-on experience across multiple areas of Android application development and software engineering:

Android Development Skills

- Understood the Android Activity lifecycle and screen navigation using Intents with role-based routing.
- Built dynamic lists with RecyclerView and a custom ContentAdapter, and used Material Design 3 components with ConstraintLayout for child-friendly layouts.
- Integrated the Text-to-Speech engine and the SpeechRecognizer to build a real listen–speak–feedback learning loop.

Data & Cloud Skills

- Designed a local SQLite schema (SQLiteOpenHelper) covering users, content, progress, preferences, and a sync queue.
- Integrated Firebase Authentication, Firestore, and Storage, plus Cloudinary and Glide for image handling.
- Implemented an offline-first architecture with a SyncManager that queues offline changes and merges local and cloud data on reconnection.

Software Engineering Skills

- Applied a Levenshtein-distance algorithm to build a practical fuzzy pronunciation-scoring engine.
- Separated concerns across Activities, models, helpers, and data managers for maintainable code.
- Collaborated using version control and divided the system into independent, integrable modules.

7.3 Challenges Faced

- Handling variability in on-device speech recognition — tuning language and silence-timeout parameters so young learners' short utterances are captured reliably.

- Keeping local SQLite data and Firestore consistent, especially reconciling changes made while offline without losing progress.
- Designing an interface simple enough for 1st and 2nd standard children while still exposing teacher tools.

7.4 Future Enhancements

- Add more standards and richer syllabus-aligned content, plus audio recordings by native speakers.
- Introduce adaptive difficulty that focuses practice on each learner's weakest words.
- Add spoken-Kannada prompts and full localisation of the interface into Kannada.
- Provide richer teacher analytics and per-student progress reports, and support additional regional languages.

7.5 References

The following resources were referred to during the design and development of the project:

- [1] Android Developers, "Text-to-Speech overview," developer.android.com/reference/android/speech/tts/TextToSpeech.
- [2] Android Developers, "SpeechRecognizer and RecognizerIntent," developer.android.com/reference/android/speech/RecognizerIntent.
- [3] Android Developers, "Save data using SQLite (SQLiteOpenHelper)," developer.android.com/training/data-storage/sqlite.
- [4] Firebase Documentation, "Firebase Authentication," firebase.google.com/docs/auth.
- [5] Firebase Documentation, "Cloud Firestore," firebase.google.com/docs/firestore.
- [6] Google Identity, "Google Sign-In for Android," developers.google.com/identity/sign-in/android.
- [7] Material Design 3 Guidelines, m3.material.io.
- [8] Cloudinary, "Android SDK documentation," cloudinary.com/documentation/android_integration.
- [9] Bumptech, "Glide image-loading library," bumptech.github.io/glide.
- [10] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions and reversals," Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.
- [11] Karnataka Government School syllabus (1st and 2nd Standard), Department of Public Instruction, Government of Karnataka.